binho

APPLICATION NOTE

Interfacing Bosch Sensortec MEMS sensors over I3C with the Binho Supernova

AN0004

Document information

Keywords	AN0004, Supernova, I3C Target Board, I3C, MIPI I3C, Bosch Sensortec, BMI323, BMP581, BMP585, Shuttle 3.0, IBI, in-band interrupt, ENTDAA, CCC, BCR, DCR, MEMS
Abstract	The current generation of Bosch Sensortec MEMS sensors offers a MIPI I3C interface alongside I2C and SPI. This application note shows how to bring one up from a Binho Supernova USB host adapter and the Binho I3C Target Board, and how to exercise the I3C features the sensor implements, without writing any firmware. It covers enumeration and dynamic address assignment, identifying a part from the bus rather than a lookup table, register access, streaming decoded sensor data, and in-band interrupts with no interrupt pin connected. Worked examples use the BMI323 inertial measurement unit and the BMP581 and BMP585 pressure sensors, and a reference Python utility that produced every output in this document is supplied.

Binho Inc.
42 Dore Street, Unit A, San Francisco, CA 94103

REVISION 2.0
1 SEPTEMBER 2026

1 Introduction

The current generation of Bosch Sensortec MEMS sensors offers a MIPI I3C interface alongside I2C and SPI, on the same two pins. I3C keeps that two-wire footprint and adds a substantial feature set on top of it.

Table 1. What I3C adds to a two-wire sensor interface

Feature	What it gives a design
Dynamic address assignment	The controller hands out addresses at run time, so static address collisions stop being a board-design constraint and identical parts can share a bus
Identification from the bus	Every target reports a 48-bit provisional identifier, a device class and its bus capabilities, so a controller can enumerate a bus it was not built for
Common command codes	A standard command set, broadcast and direct, for enumeration, identification, configuration, interrupt control and reset, the same on every compliant target
In-band interrupts	The sensor interrupts the host on the data line, carrying a byte that says why, so no interrupt pin has to be routed or allocated
Group addressing	One write reaches several targets at once, addressed as a group
Higher signalling rates	Push-pull clocking, to 12.5 MHz in single data rate mode, against the open-drain limits of I2C
Lower power	Push-pull signalling avoids the static pull-up current I2C spends continuously, and activity states let a controller tell targets what bus latency to expect so they can idle harder
Hot-join	A target powered up after the bus is running can ask to be enumerated
Timing control	The controller can have targets timestamp their interrupts against a shared reference
Standard reset and recovery	A defined target reset and defined error detection and recovery, rather than per-vendor sequences
I2C compatibility	Legacy I2C devices can share the bus

A target implements the subset of that a sensor needs, and the subset differs between parts. This note works through three Bosch Sensortec parts from a Binho Supernova USB host adapter and the Binho I3C Target Board: enumerating them, identifying them from the bus, reading and writing registers, streaming decoded measurements, getting interrupts over the data line with no interrupt pin connected, and establishing which of the features above each target actually implements.

Nothing here requires a microcontroller, firmware, or a driver port. Every step is a command from a host PC.

1.1 Scope

By the end of this document a reader with the hardware in front of them will have enumerated a sensor onto an I3C bus, identified it from its own registers, read and written registers, streamed decoded measurements, received in-band interrupts at the sensor's configured output data rate with no interrupt pin connected, and established what the target implements from observable effects.

The three parts worked through are the BMI323, BMP581 and BMP585. All three are exercised on hardware and every output in this document is a real capture.

1.2 Applicable devices

Table 2. Devices covered

Device	Function	Evaluation board	Exercised on hardware
BMI323	6-axis IMU, accelerometer and gyroscope	Shuttle Board 3.0	Yes
BMP581	Barometric pressure sensor	Shuttle Board 3.0	Yes
BMP585	Barometric pressure sensor, media resistant	Shuttle Board 3.0	Yes

The method applies to any Bosch Sensortec MEMS part with an I3C interface on a Shuttle Board 3.0 adapter. Adding one to the supplied utility is a single table entry, described in Section 11.

1.3 Related documents

Table 3. Related documents

Document	Subject
BST-BMI323-DS000-13	BMI323 datasheet
BST-BMP581-DS004-13	BMP581 datasheet
BST-BMP585-DS003-02	BMP585 datasheet
MIPI I3C Specification v1.1	Common command codes, in-band interrupts, dynamic addressing
Binho I3C Target Board product page	Sockets, jumpers, and supported adapter boards

2 Required equipment and software

2.1 Hardware

Table 4. Required equipment

Item
Binho Supernova USB host adapter
Binho I3C Target Board
A BMI323, BMP581 or BMP585 on a Shuttle Board 3.0, or another Shuttle Board 3.0 MEMS adapter
USB-C cable

The evaluation board drops into the labelled Shuttle Board 3.0 socket. Nothing else is connected for any procedure in this document.

2.2 Connections

Table 5. Signals used

Signal	Purpose
SCL	I3C clock, adapter to target
SDA	I3C data, bidirectional
3.3 V	Target supply, from the adapter
GND	Return

Two wires and a supply. **No interrupt pin, reset line or strapping resistor is connected for anything in this note**, including the interrupt sections: on I3C the sensor signals the host on SDA.

The bus runs at 3.3 V throughout. The target board provides the pull-ups.

2.3 Bus settings

Table 6. Bus settings used

Setting	Value
Push-pull rate	5 MHz at 50% duty cycle
Open-drain rate	1 MHz
Drive strength	Fast mode
Bus voltage	3.3 V

These are starting values rather than limits. Section 9 establishes what each part sustains.

2.4 Software

```
pip install binhosupernova
```

Python 3.12 and the `binhosupernova` package are the only dependencies. Unpack the assets archive that accompanies this note and confirm the setup with one command:

```
python i3c_mems.py scan
```

3 First contact: enumerate the bus

One command puts the sensor on the bus with an address the host assigned:

```
python i3c_mems.py scan
```

```
1 target(s) on the bus at 3300 mV, PUSH_PULL_5_MHZ_50_DC, OPEN_DRAIN_1_MHZ

  dynamic address 0x08
    PID  07 70 10 51 10 00   device id 0x1051
    BCR  0x06   DCR 0x62
    HDR  SDR only   IBI capable
    profile  bmp585
```

Two common command codes did that. **RSTDAA** clears any dynamic address already assigned, so the bus starts from a known state. **ENTDAA** then runs dynamic address assignment: the controller broadcasts, every target arbitrates using its 48-bit provisional identifier, and each one is handed an address in turn. With one part present that address is `0x08`.

No address was configured anywhere. The part's own static address was not used and does not need to be known.

Everything after the address in that output came from the target rather than from a lookup table, which is the first thing that feels different from I2C. On I2C a host writes an address it picked in advance and hopes something answers. Here the device introduces itself.

4 Identify the part from the bus

```
python i3c_mems.py identify --device bmp585
```

4.1 What the identity registers carry

Three registers do the introducing, and each is read with a single command rather than from the register file.

Table 7. Identity values measured

Part	PID	Device id (31:16)	BCR	DCR
BMI323	`07 70 10 43 00 00`	`0x1043`	`0x06`	`0xEF`
BMP581	`07 70 10 50 10 00`	`0x1050`	`0x06`	`0x62`
BMP585	`07 70 10 51 10 00`	`0x1051`	`0x06`	`0x62`

Every value matches the corresponding datasheet. The device identifier field embeds the chip identifier in each case, so a controller can name the part before touching the register file.

The device characteristics register says what class of device it is. Both pressure sensors report `0x62`, which is what the MIPI registry assigns to a pressure sensor, so a controller can classify a target it has no prior knowledge of. DCR identifies the class rather than the individual part, and the provisional identifier is what distinguishes one from the other.

The bus characteristics register says what the part can do on the bus. All three report `0x06`:

```
BCR 0x06
  device role                     I3C target
  advanced capabilities (bit 5)   no, so SDR only
  virtual target support          no
  offline capable                 yes
  IBI mandatory payload           yes
  IBI capable                     yes
  max data speed limit            no limit
```

Two useful facts land there before anything is configured. The part can raise in-band interrupts and will send a payload when it does, which Section 7 uses. And bit 5 is clear, so the part is single-data-rate only, which answers the high-data-rate question from the target itself in one command rather than from a datasheet table.

4.2 Confirming with the chip identifier

Table 8. Chip identifier register

Part	Register	Value read
BMI323	`0x00`	`0x1143`, low byte `0x43`
BMP581	`0x01`	`0x50`
BMP585	`0x01`	`0x51`

> **IMPORTANT** the BMI323 chip identifier register is 16 bits wide and its upper byte is a reserved field that reads `0x11` on the parts measured, so the register reads `0x1143` while the datasheet gives a reset value of `0x0043`. A host comparing the whole word against the datasheet value concludes the part is wrong. Compare the low byte.

5 Read and write registers

Register access is a private transfer: send the register address, then read the bytes.

```
python i3c_mems.py read  --device bmp585 0x01
python i3c_mems.py write --device bmi323 0x20 0x3127
```

The framing is per part rather than per vendor, and carrying an assumption from one Bosch part to another is the most likely reason a first read comes back wrong:

Table 9. Register access measured per part

Part	Register address	Data width	Dummy bytes before a read payload
BMI323	8 bits	16 bits	2
BMP581	8 bits	8 bits	0
BMP585	8 bits	8 bits	0

The BMI323 datasheet states the dummy byte count for each interface, giving two for I3C. The BMP581 and BMP585 datasheets do not state a count for I3C, and none was observed.

Where the count is not documented it can be established by reading a run of registers whose reset values are known and finding the offset at which they line up. Two adjacent anchors are enough to remove ambiguity. On the BMP585, CHIP_ID at `0x01` reads `0x51` and REV_ID at `0x02` reads `0x32`:

```
dummy=0: raw 51 32          ->  CHIP_ID 0x51 REV_ID 0x32   <== both anchors match
dummy=1: raw 51 32 22       ->  CHIP_ID 0x32 REV_ID 0x22
dummy=2: raw 51 32 22 00    ->  CHIP_ID 0x22 REV_ID 0x00
```

A single anchor would have admitted more than one offset.

The leading `0x7E` broadcast header is optional on these parts. Reads issued with it and without it returned the same value on all three, which matches the BMP581 and BMP585 datasheets: the host may skip the header and start with the dynamic address section, and this applies to all I3C transactions.

6 Stream sensor data

6.1 BMI323, accelerometer

A single write to ACC_CONF at `0x20` configures the part. The value `0x3127` selects an accelerometer range of ±8 g, an output data rate of 50 Hz, and an enabled power mode.

```
python i3c_mems.py stream --device bmi323 --seconds 5
```

Axes are 16-bit signed values at 4096 counts per g on the ±8 g range.

At rest the magnitude is 1 g, which is a free check that the whole chain is right: framing, byte order, sign handling and scale. Measured at rest the BMI323 reports 1.006 g, so the range field and the decode agree. Tilt the board and the axes redistribute while the magnitude stays put.

6.2 BMP581 and BMP585, pressure and temperature

These parts need an order rather than a single write. Configuration registers are written while the part is in standby, and only then is the measurement mode entered:

1. write ODR_CONFIG at `0x37` with a power mode of standby
2. write OSR_CONFIG at `0x36` with `press_en` set, enabling pressure alongside temperature
3. write ODR_CONFIG again with the desired output data rate and a power mode of normal
4. read OSR_EFF at `0x38` and confirm `odr_is_valid`, which reports whether the rate and oversampling combination was accepted

```
python i3c_mems.py stream --device bmp585 --seconds 2 --interval 0.5
```

```
  temperature    21.583 C   pressure 100351.7 Pa   pressure  1003.52 hPa
```

Pressure is the 24-bit unsigned value divided by 64, in pascals. Temperature is the 24-bit signed value divided by 65536, in degrees Celsius.

Ambient pressure read 1003.52 hPa on the BMP585 and 1003.88 hPa on the BMP581 in the same room within the hour, agreeing to 0.36 hPa across two parts on two boards.

The observable for these parts needs no instrument: breathe on one or lift it, and the pressure moves about 12 Pa per metre of altitude.

6.3 Output data rate

Table 10. Output data rate selection

Part	Register	Values used here
BMI323	`ACC_CONF` at `0x20`	50 Hz, as part of the `0x3127` configuration word
BMP581, BMP585	`ODR_CONFIG` at `0x37`	`0x17` 10 Hz, `0x13` 30 Hz, both listed with zero error

The rate matters in Section 7, because the interrupt rate follows it.

7 Interrupts over the bus, with no interrupt pin

In-band interrupts are the feature that changes a sensor design most visibly. The part tells the host new data is ready over the same two wires that carry the data, and the host learns why before reading anything, so no interrupt pin is routed, allocated or brought out to a connector.

7.1 Three steps

Every in-band interrupt in this note is the same three steps, in this order:

1. **Enable the source in the sensor.** The measurement itself, at an output data rate.
2. **Route it to the in-band interrupt.** Which register does this is per part, and is where most of the difficulty lives.
3. **Send ENEC with its ENINT bit.** Until this arrives the bus stays silent no matter what the sensor is doing.

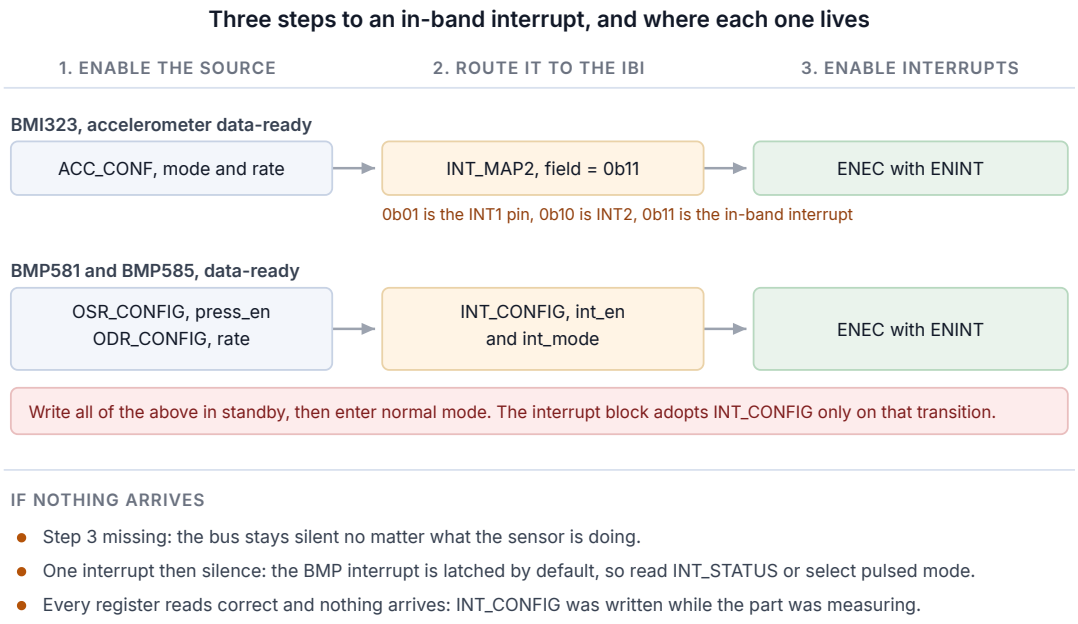

Figure 1. Three steps to an in-band interrupt, and which register carries each one on each part.

That third step is worth proving rather than asserting. Each state below was measured over a one-second window on the BMI323:

Table 11. What enables an in-band interrupt on the BMI323

State	Interrupts in 1 s
Sensor disabled	0
DISEC sent, so the target's interrupt is disabled	0
Sensor running, no routing configured	0
`INT_MAP2` written, routing configured, still disabled	0
Controller configured to accept interrupts, still disabled	0
ENEC sent with the ENINT bit	**50**

ENEC is what enables the interrupt. Neither the routing write nor the controller's own configuration does so.

```
python i3c_mems.py ibi --device bmp585 --seconds 3 --show 3
```

Table 12. Interrupt rate against configured output data rate

Part	Configured	Measured	Window
BMI323	50 Hz	50.00 per second	3.0 s
BMP581	10 Hz	10.00 per second	4.0 s
BMP585	10 Hz	9.96 per second	4.0 s
BMP585	30 Hz	29.75 per second	4.0 s

> **NOTE** the rate is comparable with the configured rate. Individual arrival times are not. Measured over 149 consecutive intervals at 50 Hz, the gaps between arrivals ranged from 0.00 to 47.00 ms with a mean of 20.03 ms, and 6 of the 149 were under 1 ms, because the host transport delivers interrupts in batches. No jitter or latency figure is given in this document for that reason.

7.2 The mandatory data byte

Each interrupt carries a mandatory data byte saying what fired, so a host can decide whether to read the sensor at all:

Table 13. Mandatory data byte observed

Part	Mandatory byte	Meaning
BMI323	`0x02`	Sample ready, accelerometer, gyroscope or temperature
BMP581, BMP585	`0x01`	Data ready

The BMI323 byte also distinguishes a FIFO watermark or full condition and the interrupt group identifier, so a host can tell new data from a FIFO event before issuing a read.

7.3 Routing, per part

BMI323. One field selects the destination. `acc_drdy_int` in INT_MAP2 at `0x3B` occupies bits 11 and 10 and takes `0b01` for the INT1 pin, `0b10` for INT2 and `0b11` for the I3C in-band interrupt. Writing `0x0C00` selects the last of these, and the interrupt then arrives over the two wires already present.

BMP581 and BMP585. INT_CONFIG at `0x14` carries the enable and the mode. Two things about it cost bench time, and both present as an interrupt setup that looks entirely correct.

> NOTE **The interrupt is latched by default.** INT_CONFIG resets to `0x35`, which selects latched mode, and INT_STATUS at `0x27` is clear-on-read. So the interrupt asserts once and is never re-armed until the host reads the status register. One interrupt and then silence is the result. Either select pulsed mode, or read INT_STATUS after each interrupt; both then give the configured rate.

> NOTE **The interrupt block adopts a new INT_CONFIG value only at the next transition from standby into a measurement mode.** The register accepts the write at any time and reads back the new value immediately, so a host that configures interrupts while the part is measuring sees every register correct and no interrupts. Measured on a BMP585 configured for 10 Hz: interrupt configuration written in normal mode, no interrupts in 2 s; the same registers followed by a standby-to-normal transition, 21 interrupts; configured in standby and then entering normal, 20 interrupts. The datasheets' own guidance covers this: write configurations before switching into the measurement mode.

7.4 Stopping the interrupt

```
python i3c_mems.py reset --device bmi323
```

DISEC with the DISINT bit stops the stream, measured as stopping after a single command on all three parts. A soft reset returns the part to its reset state, which also clears any mode a common command code has latched.

8 Exercising the rest of the I3C feature set

The utility establishes what a target implements from observable effects rather than from result codes. That distinction matters on this bus, and Figure 2 is the reason.

Table 14. The same commands at the target and at an unoccupied address

Command	At the target, `0x08`	At an unoccupied address, `0x0B`
GETPID	`SUCCESS`, returns the identifier	`I3C_NACK_ADDRESS`
GETBCR	`SUCCESS`, returns `0x06`	`I3C_NACK_ADDRESS`

A target may acknowledge a command it does not implement and discard it, which the I3C specification permits. The BMI323 datasheet marks ten common command codes as not supported and every one of them returns a successful result when sent to the part. The control above shows that is the target's behaviour rather than the adapter concealing an error: the same commands sent to an address with no target on it are refused, and the refusal is reported.

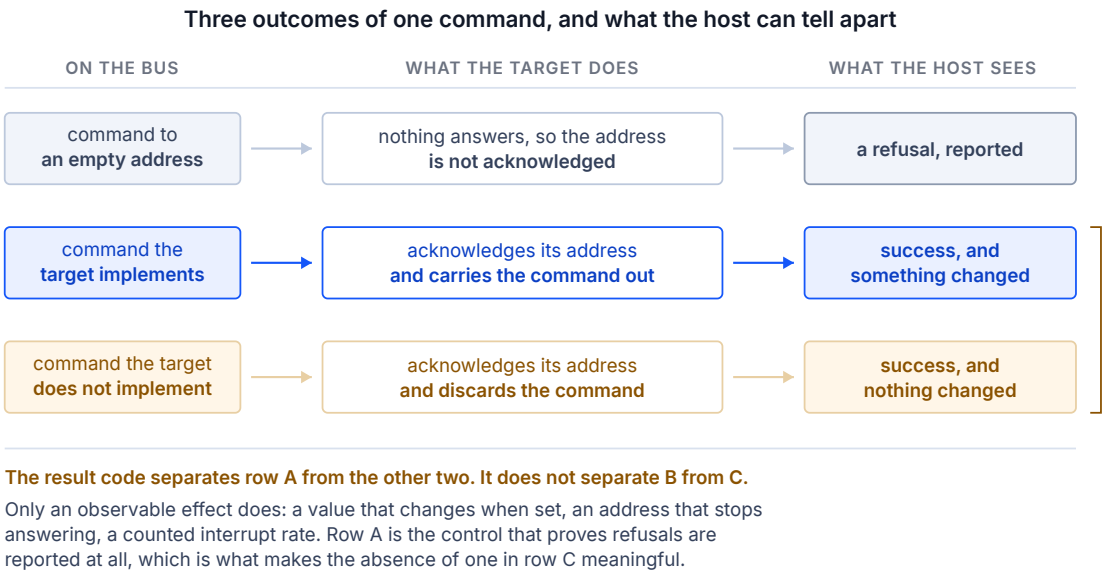

Figure 2. Three outcomes of one command, and which of them the result code can tell apart.

So each row below required an observed change: a value set and read back, an address moved and the old one confirmed to stop answering, interrupts counted against a configured rate.

```
python i3c_mems.py features --device bmp585
```

Table 15. Measured common command code support

Command	BMI323	BMP581	BMP585	Evidence
ENTDAA	Supported	Supported	Supported	Target enumerated with the expected identifier
RSTDAA	Supported	Supported	Supported	Table cleared, old address then refuses
SETNEWDA	Supported	Supported	Supported	Address moved, old address refuses
GETPID, GETBCR, GETDCR	Supported	Supported	Supported	Returned the datasheet values
ENEC, DISEC	Supported	Supported	Supported	Interrupts start and stop, repeatably
SETXTIME, GETXTIME	Supported	Supported	Supported	A reported byte changed after the command
SETMWL, SETMRL	Not implemented	Not implemented	Not implemented	Value unchanged after being set
GETMWL, GETMRL	Not implemented	Not implemented	Not implemented	As above
SETGRPA, RSTGRPA	Not implemented	Not implemented	Not implemented	A write through the group address is refused before, during and after assignment
HDR modes	Not supported	Not supported	Not supported	BCR bit 5 clear
GETHDRCAP	Not implemented	Answers `0x00`	Answers `0x00`	No HDR modes reported
Hot-join	Not observed	Not observed	Not observed	No hot-join request at any point
GETSTATUS	Undetermined	Undetermined	Undetermined	Answers, not independently checked
GETMXDS	Undetermined	Undetermined	Undetermined	Answers, no observable effect
ENTAS0 to ENTAS3	Undetermined	Undetermined	Undetermined	Needs a current measurement
RSTACT	Undetermined	Undetermined	Undetermined	No observable effect from the host

Group addressing needs its test stated, because the obvious one gives the wrong answer. A group address lets a controller address several targets at once, so it is a write destination: a read from it has no defined answer, and a target may refuse a group-address read while implementing the feature correctly. The test used here assigns a group address, writes a register through it, and reads that register back at the target's own dynamic address, and requires three results: the write refused before SETGRPA, landing while the group address is assigned, and refused again after RSTGRPA. The trial value is first written at the target's own address to confirm the register will hold it. All three parts refuse the write at every stage, at three different group addresses, while SETGRPA itself returns a successful result each time.

SETXTIME is the one row where repeating the test does not repeat the result. Its sub-commands select modes and the mode bits latch, so a sub-command the part is already in is correctly a no-op. On the BMI323, from a GETXTIME of `03 00 0D 78`, sub-command `0x3F` moves the second byte to `0x01` and never moves it again, while `0xDF` moves it to `0x03`. A host that sends one sub-command and sees no change cannot conclude the command is unimplemented. Sub-command `0xFF` disables the mode again.

Enabling timing control also changes the interrupt payload: on the BMI323 it grows from one byte to four and bit 7 of the mandatory byte is set. The mode persists across a host restart and a bus reset, and is cleared by a soft reset.

Activity states stay undetermined. Proving that a part entered a low-activity state needs a current measurement this setup cannot make, so the matrix says so rather than guessing.

9 Bus rates

```
python i3c_mems.py rates --device bmp585 --iterations 100
```

Table 16. Highest error-free bus rate

Part	Push-pull	Open drain
BMI323	12.5 MHz at 50% duty cycle	4.17 MHz
BMP581	12.5 MHz at 50% duty cycle	4.17 MHz
BMP585	12.5 MHz at 50% duty cycle	4.17 MHz

All three parts pass every rate the adapter offers. **Method:** 100 consecutive reads of the chip identifier register must all return the expected value with no error, and a rate passes only if every iteration succeeds. This is a register read loop, not a throughput measurement, and no conclusion about sustained data rate should be drawn from it.

The two rates are not independent: the adapter rejects some combinations as an invalid frequency pair, so the open-drain sweep runs against the fastest push-pull rate that passed rather than against the default.

10 Practical notes

Table 17. Practical notes

Part	Note
BMI323	The chip identifier register reads `0x1143`; compare the low byte against the datasheet's `0x43`
BMI323	Two dummy bytes precede every read payload, and data is 16 bits wide
BMP581, BMP585	Write interrupt configuration in standby, then enter normal mode, or the interrupt block never adopts it
BMP581, BMP585	The interrupt is latched by default. Read INT_STATUS after each one, or select pulsed mode
All	ENEC with ENINT is what enables the interrupt. Routing alone does not
All	The first read after enumeration may not be settled. Issue a dummy read, as the datasheets advise
All	SETXTIME latches a mode that changes the interrupt payload. `SETXTIME 0xFF` turns it off

11 The utility

Table 18. Utility commands

Command	Purpose
`scan`	Enumerate the bus and report what answered
`identify`	Decode a target's identity registers
`read`, `write`	Read or write a register, with a read-back after a write
`stream`	Print decoded samples by polling
`ibi`	Route the sensor interrupt onto the bus and count what arrives
`features`	Establish what the target implements, from observable effects
`rates`	Find the highest error-free bus rate
`reset`, `power-cycle`	Return the part to a known state

Table 19. Session options, accepted before or after the command

Option	Default
`--serial`	First adapter found
`--voltage`	3300 mV
`--push-pull`	`PUSH_PULL_5_MHZ_50_DC`
`--open-drain`	`OPEN_DRAIN_1_MHZ`

Adding a device is one entry in the profile table: the register framing, the expected identity, the writes that start a data stream, the writes that route an interrupt, and a decoder for the interrupt payload.

12 Measured results

12.1 Test configuration

Table 20. Test configuration

Item	Value
Adapter	Binho Supernova, hardware revision B, firmware 4.4.0
SDK	`binhosupernova`, Python 3.12, Windows 11
Accessory	Binho I3C Target Board
Bus	3.3 V, push-pull 5 MHz at 50% duty cycle, open drain 1 MHz, fast-mode drive strength
Enumeration	RSTDAA followed by ENTDAA
Targets	BMI323, BMP581, BMP585 on Shuttle Board 3.0, one at a time

12.2 Results

Identity. Every value in Section 4 matches the corresponding datasheet, with the BMI323 chip identifier compared on its low byte.

Interrupts. Rates as tabulated in Section 7.1. Each part was measured twice to confirm the counts are reproducible.

Support. The matrix in Section 8 was produced by the `features` command against each part, and each part was measured twice.

Bus rates. All three parts pass every rate the adapter offers, by the method stated in Section 9.

Regression coverage. The commands shown in this document are checked against recorded output for all three parts, so a change in the utility that alters what a reader would see is caught.

13 Troubleshooting

Table 21. Troubleshooting

Symptom	Check
No target enumerates	Board seated in the correct socket; 3.3 V present; try `power-cycle`
First read returns the wrong value	Dummy byte count for the part; two on the BMI323, none on the BMP parts
Chip identifier looks wrong on the BMI323	Compare the low byte; the upper byte is a reserved field
No interrupts arrive at all	That ENEC was sent with its ENINT bit
One interrupt then silence	The BMP interrupt is latched; read INT_STATUS or select pulsed mode
Every register reads correct and nothing arrives	INT_CONFIG was written while the part was measuring; write it in standby
Interrupt payload grew and bit 7 is set	Timing control is engaged; `SETXTIME 0xFF`, or soft reset
Identity reads wrongly once, then correctly	Expected after enumeration; issue a dummy read first

14 Acronyms

Table 22. Acronyms

Term	Meaning
BCR	Bus characteristics register
CCC	Common command code
DAA	Dynamic address assignment
DCR	Device characteristics register
ENTDAA	Enter dynamic address assignment
HDR	High data rate
IBI	In-band interrupt
MDB	Mandatory data byte
ODR	Output data rate
PID	Provisional identifier
SDR	Single data rate

15 Note about the source code in the document

Example code shown in this document and supplied in the accompanying archive is provided for evaluation and reference. It is released under the MIT license, the text of which is included in the archive.

Table 23. Contents of the assets archive

File	Description
`AN0004.md`	Markdown source of this document
`assets/i3c_mems.py`	Reference utility
`assets/LICENSE`	MIT license covering the example code

16 Revision history

Table 24. Revision history

Revision	Date	Description
1.0	31 August 2026	Initial release.
1.1	31 August 2026	The SETGRPA and RSTGRPA support row is re-measured with a write through the group address rather than a read from it, and the test is now stated in the document. The verdict is unchanged for all three parts. The BMI323 verdict counts are corrected, having previously been taken from a run in which the part still held a mode left behind by an earlier run.
2.0	1 September 2026	Restructured as a walkthrough, following the order a reader works in: enumerate, identify, read and write registers, stream data, interrupts, then the rest of the feature set. Section 1 now opens with what I3C adds to a two-wire sensor interface. A figure for the interrupt setup is added. Per-part behaviour that costs bench time is collected in a practical-notes section. No measured result changed.

Legal information

Definitions

Draft

A document marked draft is under internal review and subject to formal approval. Binho Inc. makes no representation or warranty as to its accuracy or completeness and accepts no liability arising from its use.

Disclaimers

Limited warranty and liability

Information in this document is believed to be accurate and reliable. Binho Inc. makes no representation or warranty, express or implied, as to its accuracy or completeness, accepts no liability for the consequences of its use, and takes no responsibility for content originating outside Binho Inc.

In no event shall Binho Inc. be liable for any indirect, incidental, punitive, special or consequential damages, including without limitation lost profits, lost savings, business interruption, or costs of removing or replacing any product, whether or not based on tort, warranty, breach of contract or any other legal theory. Notwithstanding any damages a customer might incur, the aggregate and cumulative liability of Binho Inc. is limited in accordance with its terms and conditions of commercial sale.

Right to make changes

Binho Inc. reserves the right to change the information in this document, including specifications and product descriptions, at any time and without notice. This document supersedes all information supplied prior to its publication.

Suitability for use

Binho Inc. products are not designed, authorized or warranted for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure of a Binho Inc. product can reasonably be expected to result in personal injury, death, or severe property or environmental damage. Any such inclusion or use is at the customer's own risk and Binho Inc. accepts no liability for it.

Applications

Applications described here are illustrative only. Binho Inc. makes no warranty that they are suitable for the specified use without further testing or modification. Customers are responsible for the design and operation of their own applications and products, for appropriate design and operating safeguards, and for determining whether a Binho Inc. product is suitable and fit for their purpose. Binho Inc. accepts no liability for assistance with applications or customer product design.

Third-party products and specifications

This document refers to products, boards, specifications and documentation supplied by third parties, which remain the responsibility of their respective owners. Binho Inc. makes no warranty regarding them and their behavior may change without notice. Register maps, protocol details and device identifiers reproduced here were observed on the specific hardware and firmware versions stated in this document and must be confirmed against the vendor's own documentation for the reader's part.

Example code

Source code supplied with this document is provided as an example, for evaluation and reference. It is not qualified for production use, has not been developed under a functional safety or security process, and is licensed under the terms accompanying it in the distribution archive.

Terms and conditions of commercial sale

Binho Inc. products are sold subject to the general terms and conditions of commercial sale published by Binho Inc., unless otherwise agreed in a valid written individual agreement, in which case only the terms of that agreement apply.

Export control

This document and the items it describes may be subject to export control regulations. Export may require prior authorization from the competent authorities.

Security

All products may be subject to unidentified vulnerabilities or may support security standards with known limitations. The customer is responsible for the design and operation of its applications and products throughout their lifecycle to reduce the effect of such vulnerabilities, and for compliance with all legal, regulatory and security requirements concerning its products.

Trademarks

All referenced brands, product names, service names and trademarks are the property of their respective owners. **Binho** and the Binho logo are trademarks of Binho Inc. **I3C** and **MIPI** are trademarks or registered trademarks of MIPI Alliance, Inc. **Bosch** is a registered trademark of Robert Bosch GmbH. **Bosch Sensortec** is a trademark of Bosch Sensortec GmbH.